

5.5 Optimizing Tests

AI tools can help you not only implement tests but also improve them. You've already seen an example of this when it came to implementing a whole series of tests for an array of values. But the possibilities don't end there. In the following sections, we take a look at two additional use cases: the improvement of existing tests and the implementation of missing tests.

5.5.1 Optimizing Test Code

When implementing tests, it occasionally happens that the tests correctly check the desired functionality and thus fulfill their purpose, but aren't optimally implemented. In general, the same rules apply to tests as to all other code: They should meet the quality criteria of the project. AI tools can make your work easier here by improving the code of your tests. However, this doesn't mean that you no longer need to pay attention to the quality of your source code. Rather, in this case, AI is an indicator of where there is still room for improvement.

The `UserService` TypeScript class serves as an example here. It manages an array of user objects and can add, read, and delete them. The implementation reads as follows:

```
export type User = {
  id: number;
  name: string;
};

export class UserService {
  private users: User[];

  constructor() {
    this.users = [];
  }
}
```

```

addUser(user: User): User {
  this.users.push(user);
  return user;
}
getUserById(id: number): User | undefined {
  return this.users.find(user => user.id === id);
}
deleteUserById(id: number): User | null {
  const index = this.users.findIndex(user => user.id
=== id);
  if (index !== -1) {
    return this.users.splice(index, 1)[0] || null;
  }
  return null;
}
}

```

For this class, you can write a series of tests like the following:

```

import { expect, it } from 'vitest';
import { UserService } from './UserService';
import type { User } from './UserService';

it('addUser should add a user to the list', () => {
  const userService = new UserService();
  const user: User = { id: 1, name: 'Alice' };
  userService.addUser(user);
  expect(userService['users']).toContain(user);
});

it('getUserById should return the correct user', () =>
{
  const userService = new UserService();
  const user1: User = { id: 1, name: 'Alice' };
  const user2: User = { id: 2, name: 'Bob' };
  userService.addUser(user1);
  userService.addUser(user2);
  expect(userService.getUserById(1)).toBe(user1);
  expect(userService.getUserById(2)).toBe(user2);
});

it('getUserById should return undefined for non-

```

```

existing user', () => {
  const userService = new UserService();
  expect(userService.getUserById(99)).toBeUndefined();
});

it('deleteUserById should delete the correct user', ()
=> {
  const userService = new UserService();
  const user1: User = { id: 1, name: 'Alice' };
  const user2: User = { id: 2, name: 'Bob' };
  userService.addUser(user1);
  userService.addUser(user2);
  userService.deleteUserById(1);
  expect(userService['users']).not.toContain(user1);
  expect(userService['users']).toContain(user2);
});

it('deleteUserById should return null for non-existing
user', () => {
  const userService = new UserService();
  expect(userService.deleteUserById(99)).toBeNull();
});

```

These tests cover all three cases, that is, adding, reading, and deleting. Nevertheless, the tests provide potential for improvement:

- **Duplicates with objects**

There are several instances of the `UserService` class and the user objects.

- **No use of setup and teardown routines**

The environment can be initialized before the tests and cleaned up after the tests.

- **No clean structure**

The test code is not organized, making it difficult to understand, maintain, and extend.

You can easily solve the problem with an adequate prompt. Here, the same principle applies that the more precisely you specify what you expect, the better the result will be. For example, you can indicate that you expect a clean structure, that duplicates in the code should be eliminated as much

as possible, and that the best practices of clean code should generally be followed.

Prompt: Could you please optimize the following Vitest test that is written in TypeScript? Introduce a clean structure for the tests, eliminate duplicates, and use the clean code principles.

[code of the tests]

To optimize the given tests, we can introduce a common setup function, use a consistent naming convention, and remove duplicate code. Additionally, we can make sure that each test is focused on a single behavior. Here's the refactored code:

```
import { expect, it, describe, beforeEach } from
  'vitest';
import { UserService } from './userService';
import type { User } from './userService';

describe('UserService', () => {
  let userService: UserService;
  let user1: User;
  let user2: User;

  beforeEach(() => {
    userService = new UserService();
    user1 = { id: 1, name: 'Alice' };
    user2 = { id: 2, name: 'Bob' };
  });

  it('should add a user to the list', () => {
    userService.addUser(user1);
    expect(userService['users']).toContain(user1);
  });

  it('should return the correct user by ID', () => {
    userService.addUser(user1);
    userService.addUser(user2);
    expect(userService.getUserById(1)).toBe(user1);
    expect(userService.getUserById(2)).toBe(user2);
  });
});
```

```

    it('should return undefined for non-existing user
       by ID', () => {

expect(userService.getUserById(99)).toBeUndefined();
    });

    it('should delete the correct user by ID', () => {
        userService.addUser(user1);
        userService.addUser(user2);
        userService.deleteUserById(1);
        expect(userService['users']).not.toContain(user1);
        expect(userService['users']).toContain(user2);
    });

    it('should return null when deleting a non-existing
user
       by ID', () => {
        expect(userService.deleteUserById(99)).toBeNull();
    });
});

```

[explanation of the code]

The revised tests are of significantly better quality. They are contained in a describe block that defines a context for the tests. The `userService`, `user1`, and `user2` variables are reused by the tests, and the setup routine in the `beforeEach` function ensures that the service and the user objects are instantiated correctly. This means that each test has a clean environment, and dependencies between the tests are eliminated.

Nevertheless, there is one small point of criticism. In the tests for `getUserById` and `deleteUserById`, the `addUser` method is used to write data to the service. If this method is incorrect, the other tests will also fail, whereas a test should generally only fail for one reason. A more elegant option here would be to write the information directly to the service or to transfer the data during initialization.

5.5.2 Writing Missing Tests

When developing your application, you should try to validate the entire code using automated tests. However, in some cases, this isn't possible or is too time consuming. Then, you should at least check the critical points. One tool that is very useful here is the *coverage report*. During test execution, it measures which lines, statements, and branches in the source code are checked by the tests. In general, coverage of 80% is the target value. If your work is test-driven, this value can reach up to 100%.

Most modern testing frameworks support coverage reports out of the box or as extensions. With Vitest, the framework we're using in this chapter, you can choose between the V8 and the Istanbul reporter, with the former being the default. To generate a report, you need to install the extension using the `npm install -D @vitest/coverage-v8` command and then run your tests by entering the `vitest run --coverage` command. Vitest then generates the coverage report in a series of HTML files that you can view in your browser.

As a concrete example, we use the `UserService` class from the previous example again, but with a slightly reduced range of functions. The class now only has the `addUser` and `getUserById` methods. Of the two methods, only the `addUser` method is covered by a test. [Figure 5.1](#) shows the coverage report for the file.

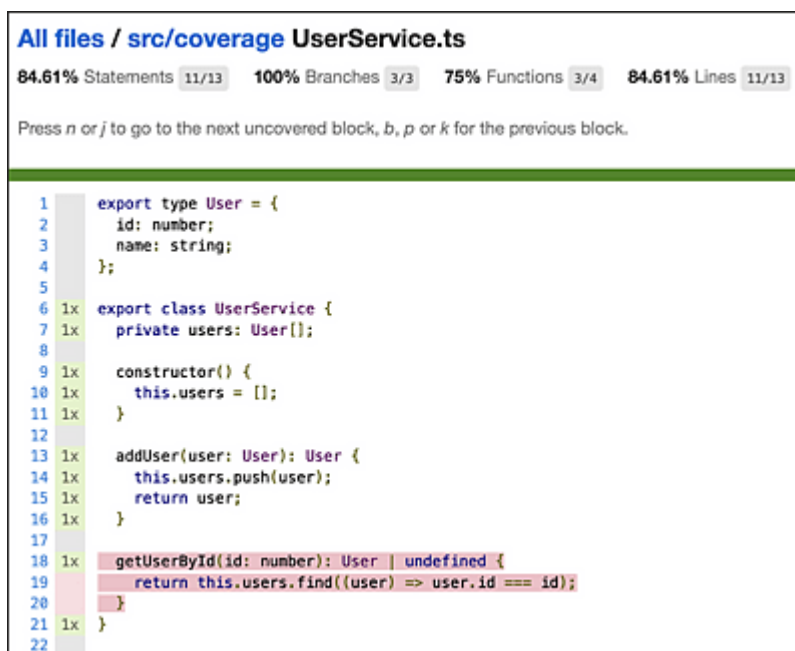


Figure 5.1 Coverage Report for the `UserService` Class

You can now transfer the HTML code of the report to ChatGPT, for example, and have the missing tests generated.

Prompt: Please generate the missing tests in Vitest using TypeScript for the following coverage report:

[code of the coverage report]

Here are the already existing tests, please extend them:

[code of the existing tests]

Here is the extended test suite for the UserService class, covering the getUserById method, which is not currently tested.

```
import { expect, it, describe, beforeEach } from
"vitest";
import { UserService } from "../UserService";
import type { User } from "../UserService";

describe("UserService", () => {
  let userService: UserService;
  let user1: User;
  let user2: User;

  beforeEach(() => {
    userService = new UserService();
    user1 = { id: 1, name: "Alice" };
    user2 = { id: 2, name: "Bob" };
  });

  it("should add a user to the list", () => {
    userService.addUser(user1);
    expect(userService["users"]).toContain(user1);
  });

  it("should return a user by ID", () => {
    userService.addUser(user1);
    const result = userService.getUserById(1);
    expect(result).toEqual(user1);
  });
});
```

```
    it("should return undefined if user ID does not  
exist", () => {  
        userService.addUser(user1);  
        const result = userService.getUserById(2);  
        expect(result).toBeUndefined();  
    });  
  
    it("should handle multiple users correctly", () => {  
        userService.addUser(user1);  
        userService.addUser(user2);  
        const result1 = userService.getUserById(1);  
        const result2 = userService.getUserById(2);  
        expect(result1).toEqual(user1);  
        expect(result2).toEqual(user2);  
    });  
});
```

The AI tool has correctly recognized from the coverage report that the `getUserById` method isn't sufficiently tested and has generated a total of three additional tests for this method. The basis for the generated tests is the existing test file, into which the new tests are seamlessly embedded. The new tests don't just call the `getUserById` method, but they also check whether it's possible to read out a user, what happens if the requested object doesn't exist, and whether it's possible to read out several users.

In this case, the AI tool makes it much easier for us to generate the tests. If, as in this example, you pass additional context in the form of existing tests, the result will be even better, and you can transfer the generated code directly into your application without making any changes.